TiDPI 07 - Rzetelny benchmark C++ i Python

Kryptonim TiDPI: Regulus

2026-10-17

Sprawozdanie TiDPI 07 - rzetelny benchmark C++ i Python

1. Kryptonim TiDPI i wykonany poziom

Kryptonim: **Regulus**. Wykonany zakres: **A–C, z uzgodnionym odstępstwem danych C**, aktualizacja 2026-10-17. B obejmuje dwie nowe pełne poprawne serie i osobną kontrolę błędną. C wykonano w dwóch kontrolowanych badaniach opisanych poniżej; D nie wykonywano. Operacja ma status uzgodnionej z prowadzącym na podstawie polecenia użytkownika: ręczna suma odległości od 500.

Jawne uzgodnienie parametrów: zastane A miało 20 020 liczb i 200 rund, a nie 10 000 liczb i 100 rund. Użytkownik wybrał zachowanie pliku A i 100 rund w B. Oczekiwanie B wynosi 501 000 000; historyczne A nadal ma 1 002 000 000. Nie porównuje się czasów A i B jako identycznych obciążeń. Użytkownik potwierdził pięć pól dokładnego nagłówka; wzmianka o sześciu polach była rozbieżnością.

Przeczytano cały szkolny PDF (10 stron), w tym A–D, artefakty, sprawozdanie i kryteria oceny. PDF dla A–B wymaga automatyzacji, walidatora, błędnej próby, dwóch serii, minimum i mediany. Dodatkowe wymagania użytkownika obejmują plan przed pomiarami, niezależne oczekiwanie, max/rozstęp, histogram kontroli ujemnej, pełne dowody i render. Lokalny AGENTS.md jest pusty; jego zastanej zmiany nie modyfikowano. Organizację raportów TiDPI_03, TiDPI_05 i TiDPI_06 wykorzystano bez przenoszenia ich wyników.

Głównym sprawozdaniem jest ten dokument. Eksport: raport PDF. Archiwum raportu A i eksportu A są niezmienionymi migawkami; ich względne odsyłacze historyczne odnoszą się do pierwotnego katalogu TiDPI_07. D01–D09 i TC-01–TC-05 zachowują historyczne znaczenie.

Aktualne pliki rozwiązania

- C++ B, Python B, generator, dane.
- Plan B, automat, walidator.
- Testy kontraktu, niezależne przeliczenie, kontrola artefaktów.
- Eksport: `sh export_report.sh`, Pandoc i XeLaTeX.

Szkolny PDF, pliki wykonywalne i pomocnicze pliki renderowania nie są częścią oddania.

Poziom C - zakres i rozstrzygnięcie danych

Wykonano dokładnie dwa rozszerzenia: **C1 Debug/Release** i **C2 rozmiar danych**. W poleceniu przyjęto błędnie, że A miało plik 10 000 liczb. Sprawdzony plik ma 20 020 liczb; jego generator tasuje cały multizbiór. Zmiana liczności przed tasowaniem zmienia prefiks mimo tego samego seedu. Użytkownik wybrał zachowanie generatora: C1 używa pierwszych 10 000 wierszy A, C2 prefiksów nowego pliku 50 000. **To jawne, zaakceptowane odstępstwo: plik 10 000 C2 nie jest identyczny z A ani z jego prefiksem 10 000. Nie deklaruje się spełnienia pierwotnego warunku tożsamości.**

Plan C rozróżnia wymagania PDF, dokładniejsze zasady użytkownika i decyzje projektowe. Lokalny AGENTS.md pozostaje pusty i niezmieniony. Audyt A–B sprawdza rzeczywiste źródła, rekordy i rozgrzewki; jest nową kontrolą dawnych dowodów, a nie historycznym uruchomieniem. Raport B i eksport zachowano w archiwum B. Migawki B, podobnie jak A, zachowują odsyłacze względem pierwotnego katalogu zadania.

Źródła C: C++, Python, przygotowanie danych, automat, walidator, niezależna kontrola. Eksport C: skrypt eksportu; z katalogu TiDPI_07: `sh rozszerzenia/C/export_C.sh`. Rozszerza reguły istniejącego eksportu B, aby podpisy grup nie oddzielały się od logów; historyczny skrypt pozostaje zachowany.

2. Cel i hipoteza

Poziom A — zapis historyczny

Pytanie: jakie czasy osiągną dwie ręczne implementacje tej samej sumy odległości, gdy dane, liczba rund i granice pomiaru są wspólne? Hipoteza zapisana w D01: obie implementacje zwrócą dokładnie wynik wzorcowy; czasy mogą się różnić, bez wybierania zwycięzcy przed pomiarem. Celem jest poprawny rdzeń benchmarku, a nie ustalenie uniwersalnej szybkości języków.

Niezależnie oczekiwany wynik: **1 002 000 000** po **200 rundach**. To wynik obliczeń; odcisk SHA-256 identyfikuje bajty pliku i ma inne znaczenie.

Poziom B — zapis historyczny

Pytanie B: czy kierunek porównania minimum i mediany dwóch implementacji powtórzy się w dwóch osobnych seriach przy wspólnym pliku i 100 rundach? Hipotezę bez wyboru zwycięzcy zapisano w D10 przed nowymi pomiarami. Obie implementacje mają uzyskać **501 000 000**. Plan B nie jest przedstawiany jako plan sprzed historycznych pomiarów A.

Poziom C - dwa pytania i hipotezy

C1: jak zestaw flag kompilacji zmienia czas przy identycznym źródle i danych? Hipoteza: konfiguracja może zmienić czas, zachowując identyczny checksum. C2: jak czas zmienia się przy 10 000, 30 000 i 50 000 elementów dla każdej implementacji przy 100 rundach? Hipoteza: wzrost jest w przybliżeniu proporcjonalny do liczby elementów. Nie zakładano zwycięzcy. Plan D17 zapisano przed pomiarami; wynik niezależny i odciski danych ustalono w D18.

3. Środowisko oraz konfiguracja

Poziom A — zapis historyczny

Metryczka D02 została zebrana 2026-10-17 o 10:22:26 czasu Europe/Warsaw. Pomiary wykonano lokalnie, kolejno, bez debuggera.

Element	Faktycznie użyta wartość (D02)
Sprzęt	Apple M4 Pro, arm64, 24 GiB RAM
System	macOS 27.0.1, build 26A434
Kompilator	Apple Clang 21.0.0, clang-2100.3.34.2
Target	arm64-apple-darwin27.0.0
Python	CPython 3.14.4
C++	C++17, Release, `-O2 -DNDEBUG`

Rzeczywiste polecenie budowania (D04), z katalogu TiDPI_07:

```
clang++ -std=c++17 -O2 -DNDEBUG -Wall -Wextra -pedantic \
  benchmark.cpp -o _build/benchmark
```

`-O2` włącza optymalizację; `-DNDEBUG` definiuje makro konfiguracji Release; standardowe `assert` nie są używane do kontroli badanego C++. Kod kompilatora wyniósł 0; oba strumienie kompilacji są puste. Nie używano Windows, Visual Studio ani MSVC. Jest to lokalna adaptacja procedury szkolnej na macOS/arm64/Clang.

C++ używa `steady_clock`, `is_steady=1`, nominalny okres typu zegara 1/1 000 000 000 s. To okres reprezentacji, nie zmierzona efektywna rozdzielczość. Tej ostatniej nie wyznaczono. Python używa `perf_counter_ns()`; `get_clock_info` zgłosił `mach_absolute_time()`, monotoniczność i rozdzielczość około **41,667 ns**. Oba programy zapisują `time_ns`; jednostka zapisu nie określa dokładności pomiaru. Informacje o zegarach zachowano na stderr (D05-D06).

Poziom B — zapis historyczny

D12 i D14 zawierają osobne aktualne metryczki. Obie serie: Apple M4 Pro, arm64, 24 GiB RAM, macOS 27.0.1 (26A434), Apple Clang 21.0.0, CPython 3.14.4. To pomiary macOS/Clang zamiast szkolnych Windows/Visual Studio/MSVC. Każdy przebieg budowano osobno, bez debuggera, z optymalizacją `-O2`, makrem `-DNDEBUG` i standardem C++17. Pełne flagi, polecenia i kody kompilacji zachowano w D12–D14.

Zegary i jednostka nie zmieniły się: C++ `steady_clock`, nominalny okres 1 ns; Python `mach_absolute_time()`, zgłoszona rozdzielczość około **41,667 ns**. Informacje są w stderr każdego procesu. Efektywnej rozdzielczości C++ i dokładności całego eksperymentu nie mierzono; jednostka `time_ns` nie dowodzi dokładności 1 ns. Wersję kompilatora zapisano bez prywatnej ścieżki instalacji.

Poziom C - rzeczywiste środowisko

D19–D21: jedna maszyna **Apple M4 Pro**, arm64, 24 GiB RAM, macOS 27.0.1 (build 26A434), Apple Clang 21.0.0 (clang-2100.3.34.2), CPython 3.14.4. Wersje i kody zapytań zapisano osobno przy każdym przebiegu. Użyto macOS/Clang zamiast Windows/Visual Studio/MSVC; nie wykonywano kroków specyficznych dla narzędzi Windows.

C1: Debug `-std=c++17 -O0 -g`, Release `-std=c++17 -O2 -DNDEBUG`. C2: ten sam zestaw Release. W C1 oba pliki binarne skompilowano z dokładnie tej samej kopii C++; różnią się wyłącznie flagami. Bez debuggera i dodatkowych flag kompilacji. Polecenia, hashe źródeł i binariów są w D19–D21.

C++ deklaruje `steady_clock`, `is_steady=1`, okres `1/1000000000` s; Python `mach_absolute_time()`, monotoniczność i rozdzielczość około **41,67 ns**. To metadane zegarów; jednostka `time_ns` nie dowodzi dokładności 1 ns. Efektywnej rozdzielczości ani narzutu odczytu zegara osobno nie zmierzono.

4. Zmienne i warunki eksperymentu

Poziom A — zapis historyczny

Czynnik porównywany: konkretna implementacja C++ lub Python. Stałe: maszyna, wspólny plik, kolejność liczb, 200 rund, ręczne pętle, 2 rozgrzewki na proces i 5 prób na blok. Łącznie każda implementacja wykonała **4 rozgrzewki i 10 prób właściwych**. Parametry ustalono przed pomiarami (D01). Rotowana jest kolejność całych bloków, nie pojedynczych prób.

Generator tworzy **20 020** liczb całkowitych z zakresu **0-1000**: każda wartość występuje dokładnie 20 razy. `random.Random(20261017).shuffle` tasuje gotowy multizbiór; **seed = 20261017**. Nie jest to niezależne losowanie każdego elementu. Plik ASCII ma jedną liczbę na wiersz i zakończenia LF. Dane zapisano raz; oba programy odczytują `dane/input.txt`.

Pełny SHA-256 w D03 i D07; poniższe dwa wiersze należy połączyć bez spacji:

```
1e1422a8679f65bb7ec778bdbd821b74a459
6431774711e89671acaa9ddca3d8
```

Weryfikator sprawdził hash bajtów i cały histogram. Hash po serii zgadzał się z hashem generatora przed kompilacją. Polecenia D04 wskazują ten sam plik; kod benchmarków tylko go odczytuje. Odrębnych danych dla języków nie generowano.

Poziom B — zapis historyczny

B zachowuje dokładnie bajty pliku A, histogram, seed i zakres opisane powyżej. Hash przed i po każdej serii jest zgodny; kopia pliku jest w każdym katalogu przebiegu. Oba procesy odczytują wspólny `dane/input.txt`. Zmieniono rounds z historycznych 200 na uzgodnione **100**, jednakowo dla obu implementacji. B bada jedną konfigurację; zmiany tej nie przedstawia się jako C.

Rotacja, rozgrzewki i próby są jak w A: blok 1 C++ → Python, blok 2 Python → C++; po 2 rozgrzewki i 5 prób na proces, czyli **4 rozgrzewki i 10 prób na implementację**. Harmonogram zapisano przed uruchomieniami. Start procesu, odczyt, alokacja, walidacja i wypisywanie są poza stoperem. Nie usuwano ani nie zastępowano czasów. Kontrola ujemna zmienia tylko środek w osobnej kopii Pythona.

Poziom C - dane i kontrolowane czynniki

Tabela 3. Dane C i niezależne checksumy po **100 rundach** (D18). Skrót SHA-256 jest odciskiem bajtów, a checksum wynikiem badanej operacji. Pełne odciski są w manifestach, nie są skracane w walidacji.

Badanie	Elementy	Oczekiwany checksum	Początek SHA-256
C1	10 000	251 042 700	`9e19f79d6545`
C2	10 000	250 039 700	`71496284c499`
C2	30 000	748 157 600	`8fd5b5c6428f`
C2	50 000	1 250 122 500	`500b7f8b59c9`

C1: prefiks historycznego A, jedyna zmienna - flagi kompilacji. C2: nadrzędny plik 50 000 utworzono przez `i % 1001` i tasowanie `random.Random(20261017).shuffle`; mniejsze pliki są jego prefiksami bajtowymi. Zakres wartości 0–1000, ASCII i LF. Przy 50 000 liczbach histogram nie ma jednakowej liczności każdej wartości. To tasowanie multizbioru, nie niezależne losowanie każdej liczby. W C2 jedynym badanym czynnikiem w obrębie implementacji jest rozmiar. Oba programy dla danego rozmiaru czytają ten sam plik. Stałe: 100 rund, kod operacji, maszyna, zegar danej implementacji i procedura prób.

Kontrola D18 potwierdziła zgodność prefiksów 10 000/30 000 z nowym plikiem 50 000 oraz **brak zgodności** 10 000 C2 z prefiksem A. Dlatego czasów C1 Release i C2 10 000 nie łączy się mimo jednakowej liczności.

5. Kontrola poprawności

Poziom A — zapis historyczny

Wzorzec niezależny od benchmarków

Dla wartości 0-1000 odległości po obu stronach 500 tworzą dwa ciągi 1-500:

```
1 + ... + 500 = 500 * 501 / 2 = 125250
jedna kopia 0..1000: 2 * 125250 = 250500
20 kopii: 20 * 250500 = 5010000
200 rund: 200 * 5010000 = 1002000000
```

Formułę zapisano przed pomiarami (D01). Po pomiarach D07 dodatkowo policzył wynik innym algorytmem: dla liczb poniżej 500 użył różnicy `500 * count - sum`, a powyżej 500 różnicy `sum - 500 * count`. Użycie bibliotecznej sumy w tej niezależnej kontroli nie zmienia ręcznego algorytmu badanego.

Implementacje i arytmetyka

W obu funkcjach wynik zaczyna od zera. Zewnętrzna pętla wykonuje 200 rund, wewnętrzna przechodzi po wspólnych danych. Oblicza `d = x - 500`, zmienia znak gdy `d < 0`, następnie dodaje d do akumulatora. Nie używa `sum()` ani `abs()` w mierzonej operacji. C++ przechowuje liczby i wynik w `int64_t`, Python w `int`.

Po walidacji wejścia d mieści się w zakresie od -500 do 500, a odległość od 0 do 500. Największy możliwy wynik wynosi `20020 * 500 * 200 = 2002000000`, znacznie mniej niż `9223372036854775807` dla `int64_t`. Iloczyny występują w uzasadnieniu, nie w badanej pętli. Akumulator rośnie monotonnie i również pośrednie wyniki mieszczą się w typie. Python ma arytmetykę całkowitą o dowolnej precyzji.

Poza stoperem każdy wynik jest porównywany ze stałą wzorcową, wypisywany lub zapisywany w diagnostyce rozgrzewki. Błędny checksum przerywa proces kodem 1; błąd wejścia/parametrów kodem 2. Samo sprawdzanie i wypisanie wyniku zapewnia obserwowalność, lecz nie wyklucza wszystkich transformacji kompilatora. Nie użyto `volatile` ani wyłączenia optymalizacji; nie badano asemblera.

Rejestr dowodów i testów

- **D01** - Plan sprzed pierwszego pomiaru; kopia w źródłach D02
- **D02** - Środowisko, SHA źródeł, kopia C++
- **D03** - Metadane generatora, dane
- **D04** - Polecenia, kody, stdout kompilatora, stderr
- **D05** - C++: blok 1 stdout, stderr, blok 2 stdout, stderr
- **D06** - Python: blok 1 stdout, stderr, blok 2 stdout, stderr

- **D07** - Weryfikacja i statystyki, stdout, stderr, kod
- **D08** - Niezależne przeliczenie
- **D09** - Kontrola końcowych artefaktów

TC-01: pełny histogram, count, seed, hash i wzorzec - **zgodne** (D03, D07). TC-02: rzeczywiste flagi Release, źródła, kod kompilatora 0 - **zgodne** (D02, D04). TC-03: nagłówek, typy, dodatnie czasy, numery 1-10, 20 rekordów i 8 rozgrzewek, checksum oraz `true`, wszystkie kody procesów 0 - **zgodne** (D05-D07). TC-04: osobne przeliczenie wzorca, median, rozstępów i wierszy 1 oraz 10 obu implementacji - **zgodne** (D08). TC-05: komplet plików, linki, anonimizacja oraz przegląd renderu eksportu - D09.

Poziom B — zapis historyczny

W B wynik jednej rundy nadal wynosi 5 010 000, po 100 rundach **501 000 000**. Walidator liczy go z liczności i sum partycji poniżej/powyżej 500, bez importowania badanej funkcji. Sprawdza też pełny histogram i stały hash danych. C++: różnica mieści się w −500…500, odległość w 0…500, a największa możliwa suma `20020 × 500 × 100 = 1001000000` mieści się w `int64_t`, także pośrednio.

Kontrakt: dokładnie `language;trial;time_ns;checksum;correct`, pięć pól; `cpp/python` zgodnie z producentem, całkowite próby i checksumy, całkowity czas ≥0, numery dokładnie 1–10 bez powtórzeń, pięć prób na blok, `correct=true` i niezależny checksum w każdym rekordzie. Oddzielnie sprawdzane są cztery rozgrzewki każdej implementacji, ich numery, czasy i checksumy. Walidator sprawdza również harmonogram, kody, konfigurację Release, informacje o zegarach i kopie źródeł.

Walidacja całości poprzedza statystyki i publikację CSV. Kody walidatora i automatu: 0 — akceptacja; 1 — odrzucenie eksperymentu; 2 — błąd obsługi/konfiguracji. Timeout ma umowny kod 124, błąd uruchomienia 127; nie wystąpiły w trzech seriach.

TC-06: integralność historycznego A — **zgodna** (D10, D15). TC-07: 18 fixture walidatora — **18 zgodnych, 0 niezgodnych** (D11). TC-08: pierwszy pełny poprawny przebieg — **zaakceptowany** (D12). TC-09: kontrola ujemna — **odrzucona, bez CSV** (D13). TC-10: drugi pełny przebieg po błędzie — **zaakceptowany** (D14). TC-11: niezależne przeliczenie i powtarzalność wniosku — **zgodne** (D15). TC-12: komplet, linki, anonimizacja i wizualny przegląd aktualnego PDF — D16.

D11 obejmuje poprawne dane, czas 0, zły nagłówek/jednostkę, brak/nadmiar pola, obcą i niezgodną implementację, zły typ, ujemny czas, duplikat, brak/nadmiar próby, `false`, błędny checksum mimo `true`, błędną rozgrzewkę, kod procesu, harmonogram i hash danych. Każde z 16 odrzuceń usuwa wcześniej istniejący poprawny CSV fixture. Są to sztuczne dane kontraktu, nie pomiary. Kod używa jawnych warunków; walidacja produkcyjna nie zależy od aktywności `assert`.

Przed błędem D13 potwierdził na rzeczywistym histogramie różne oczekiwania: `20 × 2 × 125250 × 100 = 501000000` dla środka 500, `20 × 500500 × 100 = 1001000000` dla środka 0. Nie zmieniono danych ani oczekiwania. Osobna kopia zmienia jedną linię `d = x − 500` na `d = x − 0`. Python kończy oba bloki kodem 1 już po pierwszej rozgrzewce, z `correct=false`; walidator rozpoznaje także błędny checksum i brak kompletu. Nie powstał CSV. Nie ma więc właściwych czasów Pythona z tej próby do porównania.

Poziom C - kryteria i niezależny wzorzec

Wzorzec przed pomiarami: dla wartości poniżej 500 obliczono `500 × liczność − suma`, a powyżej 500 `suma − 500 × liczność`; sumę obu części pomnożono przez 100. Nie użyto funkcji benchmarków. Po pomiarach D23 przeliczył oczekiwania drugim algorytmem, z histogramu: `suma(liczność[x] × abs(x − 500)) × 100`.

Badane funkcje wykonują ręczną pętlę 100 rund: różnica od 500, warunkowa zmiana znaku i dodanie do akumulatora. C++ używa `int64_t`, Python `int`. Maksymalnie `50000 × 500 × 100 = 2500000000`, mniej niż granica `int64_t`; różnica mieści się w −500…500, a sumy pośrednie także są bezpieczne. Wzorzec, liczność i parametry są walidowane poza stoperem.

Walidator C wymaga całego harmonogramu, identyczności danych i hashy źródeł, rzeczywistych poleceń i flag, sekwencyjnego wykonania, kodów 0, nagłówka, typów, czasów ≥0, dokładnych numerów 1–10 i dwóch rozgrzewek na proces. Każda rozgrzewka i próba musi mieć poprawny checksum oraz `true`. Dopiero po akceptacji całego badania powstają CSV i statystyki. Kontekst badania, konfiguracji, rozmiaru, rund, bloku i lokalizacji raw jest dodany w pochodnym CSV; pięciopolowy format stdout nie został zmieniony.

TC-13: integralność A–B - zgodna (D17, D23). TC-14: wzorce i prefiksy - zgodne z przyjętym rozstrzygnięciem; tożsamość z A nie zachodzi (D18). TC-15: C1 - zaakceptowane (D19). TC-16: rzeczywisty błąd - odrzucony bez

CSV (D20). TC-17: C2 po błędzie - zaakceptowane (D21). TC-18: 10 fixture walidatora - 10 zgodnych decyzji (D22). TC-19: niezależne rachunki - zgodne (D23). TC-20: końcowy komplet, linki, anonimizacja i render - D24.

6. Procedura

Poziom A — zapis historyczny

1. Przeczytano materiał i wzorce raportów, sprawdzono Git. Istniejącą zmianę pustego `AGENTS.md` pozostawiono bez modyfikacji. W TiDPI_07 nie było źródeł ani pomiarów; jedynym plikiem był szkolny PDF.
2. Zapisano D01 przed kompilacją i pomiarami; przygotowano obie implementacje.
3. Pierwsze przygotowanie metryczki przerwał sandbox przy `sysctl` (kod 1). Generator, kompilacja i pomiary jeszcze nie ruszyły. Zachowano notatkę przygotowania. Po autoryzacji wykonano serię w nowym katalogu. Notatka jest opisem zdarzenia, nie pełnym logiem ani zanonimizowaną kopią surowych czasów.
4. Zebrano D02, kopie i hashe źródeł; wygenerowano wspólny plik, zapisano D03.
5. Skompilowano C++ w Release (D04). Blok 1: C++, potem Python; blok 2: Python, potem C++. Każdy proces wykonał 2 rozgrzewki, potem 5 prób. Procesy nie działały równocześnie. Zachowano wszystkie czasy.
6. Stoper obejmuje wywołanie funkcji, inicjalizację wyniku, pętle i powrót. C++: `steady_clock::now()` i `duration_cast<nanoseconds>`; Python: `time.perf_counter_ns()`. Odczyt, alokacja i przygotowanie, walidacja wejścia, kontrola wyniku i wypisywanie pozostają poza pomiarem. Zapis czasu zawiera także nieunikniony narzut wywołania zegara.
7. Najpierw sprawdzono wszystkie rekordy i rozgrzewki. Dopiero po akceptacji wyliczono statystyki i złożono pliki `A_*_raw.txt` (D07). D08 sprawdza wybrane rachunki ponownie. Nie wykonano celowego uszkodzenia danych wymaganego w B.

Historyczne polecenia użyte dla A (bieżące źródła są już wersją B):

```
python3 collect_A.py
python3 verify_A.py wyniki/IDENTYFIKATOR_NOWEJ_SERII
```

Identyfikator wypisuje pierwszy skrypt; trzeba zastąpić nim drugą ścieżkę. Historyczny kolektor tworzył nowy katalog wyników. Nie należy go uruchamiać z bieżącymi źródłami B; odtworzenie A wymaga wersji źródeł z D02. Generator odtwarza wejście z tym samym seedem; wzorcowy hash umożliwia kontrolę zgodności bajtów. Użycie tej samej wersji CPython 3.14.4 usuwa niepewność co do zmiany procedury `shuffle` między wersjami. Można też użyć dołączonego gotowego pliku danych. Weryfikator uruchamiać zwykłym `python3`, bez –0 (korzysta z `assert`).

Poziom B — zapis historyczny

1. Sprawdzono Git, zarchiwizowano raport i PDF A oraz zweryfikowano hashe historycznych kopii. Zapisano plan B przed nowymi pomiarami (D10).
2. Zmieniono obie implementacje do 100 rund i wykonano testy walidatora (D11).
3. Pierwsze przygotowanie B przerwał odczyt metryczki w sandboxie; katalog niepełnego przygotowania nie zawiera kompilacji ani pomiarów. Po zezwoleniu wykonano nową serię D12.
4. Automat zapisał harmonogram, kopie źródeł i dane, metryczkę, skompilował świeży plik w katalogu przebiegu i uruchomił cztery procesy kolejno. Każdy strumień i kod zapisano oddzielnie, również przy błędzie. Limit procesu wynosi 60 s, kompilacji 120 s. Po nieudanej kompilacji nie uruchamia się programu.
5. Zapisano histogram D13, utworzono uszkodzoną kopię i wykonano rzeczywiste odrzucenie. Automat zachował także poprawne procesy C++, bez analizowania ich jako zaakceptowanej serii.
6. Uruchomiono ponownie poprawne źródła w D14. Oryginał Pythona pozostał niezmieniony przez kontrolę ujemną; jego hash odpowiada poprawnemu D14.
7. Niezależnie przeliczono wyniki (D15), uzupełniono raport i wykonano eksport (D16).

Odtworzenie B z katalogu zadania (każda komenda tworzy nowy katalog):

```
python3 test_validate_results.py
python3 run_benchmark.py --label good1
python3 run_benchmark.py --label fault \
  --python-source kontrola_bledna/benchmark.py
python3 run_benchmark.py --label good2
python3 validate_results.py przebiegi/IDENTYFIKATOR
```

Przebieg `fault` ma oczekiwany kod 1. Wymagane są lokalne Clang, Python, `sw_vers` i `sysctl`. Nie uruchamiać serii równolegle. Plan i kontrola histogramu są zachowane w załącznikach; `verify_B.py` przelicza

wskazane serie historyczne.

Poziom C - wykonanie i harmonogram

1. Zarchiwizowano raport B, zapisano hashe zastanych plików. Plan i harmonogramy utrwalono o 11:03:47 dnia 2026-10-17 (Europe/Warsaw), przed pomiarami C.
2. Przygotowano dane i wzorce D18. D19: pięć bloków C1, w każdym proces Debug i Release, każdy z dwiema rozgrzewkami i dwiema próbami. Nieparzyste bloki zaczyna Debug, parzyste Release. Numery prób kolejno 1–2 aż do 9–10.
3. D20: uruchomiono pełny zaplanowany harmonogram C2 z osobną kopią Pythona, w której zmieniono `x-500` na `x-0`. Każdy wadliwy proces przerwał na pierwszej rozgrzewce. Zachowano także wszystkie wykonane pomiary C++, lecz całe badanie odrzucono i nie analizowano jego czasów.
4. D21: poprawny C2 po kontroli błędnej. Kolejności rozmiarów w blokach: 10–30–50, 30–50–10, 50–10–30, 10–30–50, 30–50–10 (tysięcy). W każdym rozmiarze bloki nieparzyste zaczyna C++, parzyste Python. Każdy proces wykonuje dwie rozgrzewki i dwie próby. Rotacja dotyczy bloków.
5. D22–D23: fixture i niezależne przeliczenie. D24: aktualizacja raportu, eksport, kontrola powiązań i przegląd wyrenderowanych stron.

W poprawnym C1 wykonano **10 procesów, 20 rozgrzewek i 20 prób**; w poprawnym C2 **30 procesów, 60 rozgrzewek i 60 prób**. Na każdą grupę przypada 10 właściwych prób i 10 rozgrzewek. To jedna zaakceptowana seria każdego badania, oddzielna od historycznych dwóch serii B. Niczego nie uruchamiano równolegle w harmonogramie. Nie usuwano niewygodnych czasów ani nie zastępowano ich nowymi próbami.

Pomiar: start zegara → wywołanie funkcji → stop zegara. Obejmuje inicjalizację akumulatora i obie pętle; wyklucza uruchomienie procesu, odczyt, przygotowanie, walidację, porównanie wyniku i wypisywanie. Obserwowalny checksum nie wyklucza wszystkich transformacji optymalizatora; nie badano asemblera.

Pierwsza próba C1 (niepełny przebieg) zatrzymała się na odmowie `sysctl` przez piaskownicę: bez kompilacji i pomiarów. Późniejsza kontrola odrzuciła niekompletny zbiór. Po dopuszczeniu wykonania poza piaskownicą powstał nowy D19. Niepełnych strumieni nie poprawiano.

Odtworzenie, z katalogu `rozszerzenia/C`, po przygotowaniu danych:

```
python3 run_C.py C1
python3 run_C.py C2 --fault
python3 run_C.py C2
python3 test_C.py C2/przebiegi/ID_POPRAWNEGO_PRZEBIEGU
python3 verify_C.py
```

`ID_POPRAWNEGO_PRZEBIEGU` zastępuje się nazwą wypisaną przez automat. `prepare_C.py` odtwarza dane w świeżym zestawie i chroni istniejące dowody przed nadpisaniem. `verify_C.py` jest kontrolą tego zestawu: oczekuje po jednym zaakceptowanym przebiegu każdego badania; kolejne powtórzenia analizuje się osobno.

7. Surowe wyniki

Poziom A — zapis historyczny

D05 i D06 to pełne, niezmienione strumienie procesów. Rozgrzewki są osobnymi rekordami na stderr i nie należą do tabeli prób. Poniżej pełna zawartość pochodnych plików `A_cpp_raw.txt` i `A_python_raw.txt`: połączono po dwa bloki, zachowując wszystkie pola i jeden nagłówek. Nie przedstawia się ich jako stdout pojedynczego procesu. Oryginalne bloki pozostają do kontroli.

C++ (D05, agregacja D07):

```
language;trial;time_ns;checksum;correct
cpp;1;808500;1002000000;true
cpp;2;795375;1002000000;true
cpp;3;773084;1002000000;true
cpp;4;840916;1002000000;true
cpp;5;779375;1002000000;true
cpp;6;668208;1002000000;true
cpp;7;730208;1002000000;true
cpp;8;731834;1002000000;true
cpp;9;676125;1002000000;true
cpp;10;659875;1002000000;true
```

Python (D06, agregacja D07):

```
language;trial;time_ns;checksum;correct
python;1;107626500;1002000000;true
python;2;106623500;1002000000;true
python;3;107586333;1002000000;true
python;4;106552750;1002000000;true
python;5;106012416;1002000000;true
python;6;133934750;1002000000;true
python;7;139381458;1002000000;true
python;8;132183500;1002000000;true
python;9;135451250;1002000000;true
python;10;135279125;1002000000;true
```

Poziom B — zapis historyczny

Rejestr nowych dowodów B

- **D10** — plan B, integralność i archiwizacja A.
- **D11** — 18 testów walidatora, źródło testów.
- **D12** — przebieg 1: metryczka, harmonogram, polecenia, kody, walidacja, zaakceptowany CSV.
- **D13** — kontrola błędna: histogram i oczekiwania, uszkodzone źródło, diff, metryczka, harmonogram, polecenia, kody, odrzucenie.
- **D14** — przebieg 2: metryczka, harmonogram, polecenia, kody, walidacja, zaakceptowany CSV.
- **D15** — niezależne rachunki, skrypt kontroli.
- **D16** — końcowa kontrola artefaktów.

W każdym D12–D14 `block1_*` i `block2_*` są niezmienionymi strumieniami poszczególnych procesów, nie agregacją. `exit_codes.txt` zawiera także kod walidatora; `exit_codes.json` zawiera kody kompilacji i benchmarków. `execution.json` zapisuje rzeczywisty czas i status wywołań. `sources/` i `source_sha256.json` identyfikują kod. `input.txt` jest kopią wspólnego wejścia. `validate_stdout.txt` i `validate_stderr.txt` są strumieniami walidatora. Nie redagowano surowych dowodów.

D12 — komplet strumieni:

- Blok 1: cpp stdout, cpp stderr, python stdout, python stderr.
- Blok 2: cpp stdout, cpp stderr, python stdout, python stderr.
- kompilacja stdout, kompilacja stderr, stdout walidatora, stderr walidatora, hashe źródeł.

D13 — komplet strumieni:

- Blok 1: cpp stdout, cpp stderr, python stdout, python stderr.
- Blok 2: cpp stdout, cpp stderr, python stdout, python stderr.

- kompilacja stdout, kompilacja stderr, stdout walidatora, stderr walidatora, hashe źródeł.

D14 — komplet strumieni:

- Blok 1: cpp stdout, cpp stderr, python stdout, python stderr.
- Blok 2: cpp stdout, cpp stderr, python stdout, python stderr.
- kompilacja stdout, kompilacja stderr, stdout walidatora, stderr walidatora, hashe źródeł.

Poniżej pełne rekordy właściwych prób B jako pochodna prezentacja bloków; oryginalne stdout są wskazane powyżej.

D12 — przebieg 1

cpp:

```
language;trial;time_ns;checksum;correct
cpp;1;552000;501000000;true
cpp;2;464959;501000000;true
cpp;3;467875;501000000;true
cpp;4;463000;501000000;true
cpp;5;459292;501000000;true
cpp;6;335875;501000000;true
cpp;7;339458;501000000;true
cpp;8;341750;501000000;true
cpp;9;335541;501000000;true
cpp;10;303792;501000000;true
```

python:

```
language;trial;time_ns;checksum;correct
python;1;54592709;501000000;true
python;2;55922333;501000000;true
python;3;55462792;501000000;true
python;4;55978709;501000000;true
python;5;54561709;501000000;true
python;6;54863041;501000000;true
python;7;55469000;501000000;true
python;8;57055084;501000000;true
python;9;55348208;501000000;true
python;10;55618583;501000000;true
```

D14 — przebieg 2

cpp:

```
language;trial;time_ns;checksum;correct
cpp;1;352333;501000000;true
cpp;2;353416;501000000;true
cpp;3;353292;501000000;true
cpp;4;376042;501000000;true
cpp;5;373375;501000000;true
cpp;6;306958;501000000;true
cpp;7;306125;501000000;true
cpp;8;323375;501000000;true
cpp;9;340042;501000000;true
cpp;10;319292;501000000;true
```

python:

```
language;trial;time_ns;checksum;correct
python;1;57780875;501000000;true
python;2;57082500;501000000;true
python;3;57581083;501000000;true
python;4;56790667;501000000;true
python;5;57419042;501000000;true
python;6;57217875;501000000;true
python;7;57593208;501000000;true
python;8;57442667;501000000;true
python;9;58419209;501000000;true
python;10;56792625;501000000;true
```

D13 — fragment błędnego stderr Pythona

`warmup=1;time_ns=37196334;checksum=1001000000;correct=false`

To fragment pierwszego procesu, nie pełny stderr. Walidator ma kod 1, `accepted=false`, `csv_created=false`, `published_records=0`.

Poziom C - rejestr dowodów

- **D17** - Plan i zachowanie podstaw: plan, czas i hashe planu, audyt A–B, manifest zastanych plików.
- **D18** - Dane i niezależne oczekiwania: manifest C1, manifest C2, porównanie prefiksów.
- **D19** - Poprawny C1: metryczka, harmonogram, polecenia, kody, walidacja, CSV.
- **D20** - Rzeczywista kontrola błędna: zmieniony Python, metryczka, harmonogram, polecenia, kody, odrzucenie.
- **D21** - Poprawny C2 po błędzie: metryczka, harmonogram, polecenia, kody, walidacja, CSV.
- **D22** - Testy walidatora: 10 przypadków, źródło testów.
- **D23** - Niezależna kontrola: rachunki, źródło.
- **D24** - Kontrola końcowa: wynik, weryfikator.

W każdym przebiegu pliki `bB_N_W_stdout.txt` i `bB_N_W_stderr.txt` są pełnymi, niezmienionymi strumieniami procesu: B to blok, N to liczność, W to wariant. `execution.json` zapisuje kolejność i czasy, `sources` zawiera dokładne wersje źródeł, `source_sha256.json` ich hashe, `data` kopie wspólnych wejść.

Poniżej bezpośrednie odsyłacze do wszystkich strumieni benchmarków; każdy wpis obejmuje stdout i stderr, także w odrzuconym D20.

- D19 `b1_10000_Debug`: stdout, stderr.
- D19 `b1_10000_Release`: stdout, stderr.
- D19 `b2_10000_Release`: stdout, stderr.
- D19 `b2_10000_Debug`: stdout, stderr.
- D19 `b3_10000_Debug`: stdout, stderr.
- D19 `b3_10000_Release`: stdout, stderr.
- D19 `b4_10000_Release`: stdout, stderr.
- D19 `b4_10000_Debug`: stdout, stderr.
- D19 `b5_10000_Debug`: stdout, stderr.
- D19 `b5_10000_Release`: stdout, stderr.
- D20 `b1_10000_cpp`: stdout, stderr.
- D20 `b1_10000_python`: stdout, stderr.
- D20 `b1_30000_cpp`: stdout, stderr.
- D20 `b1_30000_python`: stdout, stderr.
- D20 `b1_50000_cpp`: stdout, stderr.
- D20 `b1_50000_python`: stdout, stderr.
- D20 `b2_30000_python`: stdout, stderr.
- D20 `b2_30000_cpp`: stdout, stderr.
- D20 `b2_50000_python`: stdout, stderr.
- D20 `b2_50000_cpp`: stdout, stderr.
- D20 `b2_10000_python`: stdout, stderr.
- D20 `b2_10000_cpp`: stdout, stderr.
- D20 `b3_50000_cpp`: stdout, stderr.

- D20 b3_50000_python: stdout, stderr.
- D20 b3_10000_cpp: stdout, stderr.
- D20 b3_10000_python: stdout, stderr.
- D20 b3_30000_cpp: stdout, stderr.
- D20 b3_30000_python: stdout, stderr.
- D20 b4_10000_python: stdout, stderr.
- D20 b4_10000_cpp: stdout, stderr.
- D20 b4_30000_python: stdout, stderr.
- D20 b4_30000_cpp: stdout, stderr.
- D20 b4_50000_python: stdout, stderr.
- D20 b4_50000_cpp: stdout, stderr.
- D20 b5_30000_cpp: stdout, stderr.
- D20 b5_30000_python: stdout, stderr.
- D20 b5_50000_cpp: stdout, stderr.
- D20 b5_50000_python: stdout, stderr.
- D20 b5_10000_cpp: stdout, stderr.
- D20 b5_10000_python: stdout, stderr.
- D21 b1_10000_cpp: stdout, stderr.
- D21 b1_10000_python: stdout, stderr.
- D21 b1_30000_cpp: stdout, stderr.
- D21 b1_30000_python: stdout, stderr.
- D21 b1_50000_cpp: stdout, stderr.
- D21 b1_50000_python: stdout, stderr.
- D21 b2_30000_python: stdout, stderr.
- D21 b2_30000_cpp: stdout, stderr.
- D21 b2_50000_python: stdout, stderr.
- D21 b2_50000_cpp: stdout, stderr.
- D21 b2_10000_python: stdout, stderr.
- D21 b2_10000_cpp: stdout, stderr.
- D21 b3_50000_cpp: stdout, stderr.
- D21 b3_50000_python: stdout, stderr.
- D21 b3_10000_cpp: stdout, stderr.
- D21 b3_10000_python: stdout, stderr.
- D21 b3_30000_cpp: stdout, stderr.
- D21 b3_30000_python: stdout, stderr.
- D21 b4_10000_python: stdout, stderr.
- D21 b4_10000_cpp: stdout, stderr.
- D21 b4_30000_python: stdout, stderr.
- D21 b4_30000_cpp: stdout, stderr.
- D21 b4_50000_python: stdout, stderr.
- D21 b4_50000_cpp: stdout, stderr.
- D21 b5_30000_cpp: stdout, stderr.
- D21 b5_30000_python: stdout, stderr.
- D21 b5_50000_cpp: stdout, stderr.
- D21 b5_50000_python: stdout, stderr.
- D21 b5_10000_cpp: stdout, stderr.
- D21 b5_10000_python: stdout, stderr.

Poziom C - komplet właściwych rekordów zaakceptowanych serii

Poniższe bloki to agregacje pełnych rekordów stdout, rozdzielone według badania, konfiguracji i rozmiaru; pierwotne strumienie z własnymi nagłówkami są w D19/D21. Rozgrzewki pozostają oddzielnie w stderr.

C1, Debug, 10000 elementów, cpp, 100 rund:

```
language;trial;time_ns;checksum;correct
cpp;1;10899417;251042700;true
cpp;2;9806583;251042700;true
cpp;3;9198291;251042700;true
cpp;4;9106500;251042700;true
cpp;5;7857916;251042700;true
cpp;6;8072333;251042700;true
cpp;7;8182375;251042700;true
cpp;8;7706292;251042700;true
cpp;9;7253959;251042700;true
cpp;10;7628917;251042700;true
```

C1, Release, 10000 elementów, cpp, 100 rund:

```
language;trial;time_ns;checksum;correct
cpp;1;365083;251042700;true
cpp;2;356792;251042700;true
cpp;3;291958;251042700;true
cpp;4;279625;251042700;true
cpp;5;167750;251042700;true
cpp;6;167542;251042700;true
cpp;7;149000;251042700;true
cpp;8;150041;251042700;true
cpp;9;169417;251042700;true
cpp;10;184042;251042700;true
```

C2, CPython, 10000 elementów, python, 100 rund:

```
language;trial;time_ns;checksum;correct
python;1;27792917;250039700;true
python;2;27906459;250039700;true
python;3;35362166;250039700;true
python;4;34936333;250039700;true
python;5;28126125;250039700;true
python;6;27582875;250039700;true
python;7;34426083;250039700;true
python;8;34716416;250039700;true
python;9;29058042;250039700;true
python;10;39755250;250039700;true
```

C2, CPython, 30000 elementów, python, 100 rund:

```
language;trial;time_ns;checksum;correct
python;1;105892250;748157600;true
python;2;106783875;748157600;true
python;3;83214167;748157600;true
python;4;83409458;748157600;true
python;5;83147375;748157600;true
python;6;98458542;748157600;true
python;7;111272375;748157600;true
python;8;95245333;748157600;true
python;9;84061458;748157600;true
python;10;85746541;748157600;true
```

C2, CPython, 50000 elementów, python, 100 rund:

```
language;trial;time_ns;checksum;correct
python;1;145687417;1250122500;true
python;2;146225375;1250122500;true
python;3;142522958;1250122500;true
python;4;142815458;1250122500;true
python;5;148166834;1250122500;true
python;6;144019583;1250122500;true
python;7;141958000;1250122500;true
python;8;143600042;1250122500;true
python;9;185816708;1250122500;true
python;10;178493208;1250122500;true
```

C2, Release, 10000 elementów, cpp, 100 rund:

```
language;trial;time_ns;checksum;correct
cpp;1;148916;250039700;true
cpp;2;148917;250039700;true
cpp;3;170709;250039700;true
cpp;4;170792;250039700;true
cpp;5;149042;250039700;true
cpp;6;151750;250039700;true
cpp;7;161625;250039700;true
cpp;8;149042;250039700;true
cpp;9;202334;250039700;true
cpp;10;474084;250039700;true
```

C2, Release, 30000 elementów, cpp, 100 rund:

```
language;trial;time_ns;checksum;correct
cpp;1;509708;748157600;true
cpp;2;532709;748157600;true
cpp;3;446792;748157600;true
cpp;4;446750;748157600;true
cpp;5;475542;748157600;true
cpp;6;446792;748157600;true
cpp;7;512875;748157600;true
cpp;8;512834;748157600;true
cpp;9;500666;748157600;true
cpp;10;489917;748157600;true
```

C2, Release, 50000 elementów, cpp, 100 rund:

```
language;trial;time_ns;checksum;correct
cpp;1;778250;1250122500;true
cpp;2;777416;1250122500;true
cpp;3;842708;1250122500;true
cpp;4;825500;1250122500;true
cpp;5;856292;1250122500;true
cpp;6;852834;1250122500;true
cpp;7;792292;1250122500;true
cpp;8;746375;1250122500;true
cpp;9;776250;1250122500;true
cpp;10;803500;1250122500;true
```

Poziom C - kontrola ujemna i fixture

D20: rzeczywisty fragment stderr pierwszego wadliwego procesu Pythona:

```
warmup=1;time_ns=35027208;checksum=503213100;correct=false
```

Dla 10 000 C2 oczekiwano 250039700, więc kod procesu wynosi 1, a walidator odrzucił cały zbiór bez CSV i statystyk. Oryginalny Python pozostał poprawny; D21 potwierdza zgodność jego hashy i działanie po błędzie. D23 sprawdza także, że uszkodzony wariant różni się właśnie środkiem obliczenia.

D22 obejmuje komplet poprawny oraz odrzucenia: checksum przy `true`, brak próby, duplikat próby, zła liczność w manifeście, złe flagi, brak rozgrzewki, jednostka `time_us`, zmieniony harmonogram i niezerowy kod procesu. Wynik: **10 zgodnych, 0 niezgodnych decyzji**. Dziewięć odrzuconych fixture miało wcześniej poprawny CSV; po odrzuceniu plik nie istnieje. Są to oznaczone kopie testowe, a nie kolejne pomiary wydajności.

8. Opracowanie wyników

Poziom A — zapis historyczny

Tabela 1. Jedna seria A, ta sama konfiguracja, 10 prób na implementację; jednostka **ms**, przeliczenie `time_ns / 1000000`. Źródło: D05-D08.

Miara	C++ Release	Python
Minimum	0,659875	106,012416
Maksimum	0,840916	139,381458
Rozstęp	0,181041	33,369042
Mediana	0,752459	119,905000

Mediana przy 10 próbach to średnia piątej i szóstej wartości po sortowaniu. Dla C++: `(731834 + 773084) / 2 = 752459 ns`. Dla Python: `(107626500 + 132183500) / 2 = 119905000 ns`. Rozstęp to maksimum minus minimum. Nie zaokrąglano danych przed obliczeniem. Minimum pokazuje najszybszą zaobserwowaną próbę, mediana środek rozkładu zachowanych czasów. Żadna z nich nie jest gwarantowanym czasem programu.

D08 ponownie sprawdza te rachunki z użyciem `Decimal`, a także surowe wiersze 1 i 10 obu języków: checksum zgodny z formułą i przeliczenie ns na ms. Przykładowo pierwsza próba C++: `808500 ns = 0,808500 ms`; pierwsza Python: `107626500 ns = 107,626500 ms`. Są to kontrole asystenta; nie deklaruje się ręcznej weryfikacji ucznia.

Poziom B — zapis historyczny

Tabela 2. Poprawne serie B, 100 rund, po 10 prób na implementację; **ms**. Źródła: D12, D14, niezależna kontrola D15.

Seria	Implementacja	Minimum	Maksimum	Rozstęp	Mediana
1	cpp	0,303792	0,552	0,248208	0,400521
1	python	54,561709	57,055084	2,493375	55,465896
2	cpp	0,306125	0,376042	0,069917	0,3461875
2	python	56,790667	58,419209	1,628542	57,4308545

Mediana jest średnią pozycji 5 i 6 po sortowaniu. W serii 1: C++ `(341750 + 459292) / 2 = 400521 ns`, Python `(55462792 + 55469000) / 2 = 55465896 ns`. W serii 2: C++ `(340042 + 352333) / 2 = 346187,5 ns`, Python `(57419042 + 57442667) / 2 = 57430854,5 ns`. Zachowano połówki ns wynikające z rachunku; nie oznaczają większej dokładności zegara. Rozstępy to różnice skrajnych wartości, bez wstępnego zaokrąglania.

D15 innym odczytem CSV i rachunkiem `Decimal` porównał wszystkie rekordy CSV z oryginalnymi blokami oraz ponownie obliczył min/max/rozstęp/medianę dla czterech grup. Osobno sprawdził checksum i przeliczenie ns→ms w próbach 1 i 10 obu implementacji w obu seriach: osiem wierszy. Weryfikację wykonał asystent.

Minimum odpowiada na pytanie o najlepszy zaobserwowany czas w tej serii. Mediana opisuje środek uporządkowanych czasów i słabiej reaguje na pojedyncze skrajne obserwacje. Minimum nie jest gwarantowanym osiągalnym czasem, a mediana nie dowodzi stabilności wszystkich prób. Przy dwóch grupach czasów może leżeć między nimi, co widać także w historycznym A. Błędnej serii nie podsumowano statystycznie.

Poziom C - statystyki osobnych grup

Tabela 4. Jednostka **ms**, po 10 właściwych prób w każdej grupie; D19/D21, niezależne przeliczenie D23. Nie łączono badań ani rozmiarów.

Badanie	Konfiguracja	Elementy	Minimum	Maksimum	Rozstęp	Mediana
C1	Debug	10000	7,253959	10,899417	3,645458	8,127354
C1	Release	10000	0,149	0,365083	0,216083	0,1767295
C2	CPython	10000	27,582875	39,75525	12,172375	31,7420625
C2	CPython	30000	83,147375	111,272375	28,125	90,495937
C2	CPython	50000	141,958	185,816708	43,858708	144,8535
C2	Release	10000	0,148916	0,474084	0,325168	0,1566875
C2	Release	30000	0,44675	0,532709	0,085959	0,4952915
C2	Release	50000	0,746375	0,856292	0,109917	0,797896

C1: iloraz median Debug/Release = **45,99** (D19, D23).

Tabela 5. C2: stosunek median do 10 000 oraz mediana podzielona przez liczbę elementów, w ns/element dla całej próby obejmującej 100 rund (D21, D23).

Implementacja	Elementy	Mediana / mediana 10 000	ns/element
cpp	10000	1	15,669
cpp	30000	3,161	16,51
cpp	50000	5,092	15,958
python	10000	1	3174,206
python	30000	2,851	3016,531
python	50000	4,563	2897,07

Ilorazy w Tabeli 5 i ns/element zaokrąglono do trzech miejsc; Tabela 4 zachowuje wartości wynikające z ns, w tym połówki ns median. D23 ponownie odczytał wszystkie surowe wiersze i porównał je z CSV, przeliczył wszystkie minima, maksima, rozstępy i mediany. Osobno zapisał wiersze **1 i 10** każdej z ośmiu grup wraz z przeliczeniem ns→ms oraz pary pozycji 5/6. Przykład C1 Debug: `(8072333 + 8182375) / 2 = 8127354 ns`; Release: `(169417 + 184042) / 2 = 176729,5 ns`. Minimum opisuje najkrótszą zaobserwowaną próbę, mediana środkowy poziom serii; żadna z tych miar nie jest gwarancją czasu bez zakłóceń.

9. Interpretacja

Poziom A — zapis historyczny

Obserwacja: w tej serii C++ Release miał niższe minimum i medianę niż ręczna implementacja Python. Obie zwróciły poprawne wyniki we wszystkich próbach oraz rozgrzewkach. Dotyczy to wskazanego pliku, 200 rund i użytych programów.

Obserwacja o kolejności (D08): wszystkie czasy C++ z bloku 2 (0,659875-0,731834 ms) były niższe niż z bloku 1 (0,773084-0,840916 ms). W Pythonie blok 2 (132,183500-139,381458 ms) był wolniejszy niż blok 1 (106,012416-107,626500 ms). Zachowano oba bloki. Zbiorcza mediana Pythona leży pomiędzy grupami, dlatego nie opisuje czasu pojedynczego typowego bloku.

Przyczyny różnic między blokami nie ustalono. Nie mierzono temperatury, częstotliwości, cache ani obciążenia tła. Rotacja kolejności ogranicza stałe uprzywilejowanie miejsca w harmonogramie, ale nie zapewnia identycznych warunków. Sama obserwacja nie pozwala przypisać różnic konkretnemu mechanizmowi sprzętu ani kosztowi interpretera.

Poziom B — zapis historyczny

Obserwacja B: w obu poprawnych seriach minimum i mediana C++ Release są niższe niż Pythona. Kierunek wniosku powtórzył się, chociaż wartości i rozstępy nie są identyczne: mediana C++ wyniosła 0,400521 i 0,3461875 ms, Pythona 55,465896 i 57,4308545 ms. W obu seriach również maksimum C++ jest poniżej minimum Pythona. To obserwacja tych programów, pliku i konfiguracji.

Obserwacja kontroli ujemnej: sam szybki wynik nie wystarcza. Uszkodzona kopia rzeczywiście wykonała inną operację, została odrzucona niezależnie od czasu, a późniejsza poprawna seria została zaakceptowana. Zgodność samych dwóch producentów nie zastępuje wzorca.

Nie ustalono przyczyn zmian czasów między seriami. Nie mierzono cache, temperatury, taktowania ani obciążenia. Nie wyprowadza się uniwersalnego współczynnika szybkości języków ani przyczyn związanych z interpreterem.

Poziom C - obserwacje i granice interpretacji

C1, obserwacja: przy wspólnym prefiksie A, 100 rundach i identycznym źródle Release miał niższą medianę niż Debug; iloraz median wyniósł 45,99. Wszystkie wyniki obliczeń pozostały identyczne. Jest to wynik dwóch zestawów flag w tym eksperymencie; nie rozdziela wpływu poszczególnych flag ani mechanizmów optymalizatora. Nie badano asemblera, cache ani temperatury.

C2, obserwacja: mediany rosną dla obu implementacji wraz z licznością. Dla trzykrotnego i pięciokrotnego rozmiaru stosunki wyniosły odpowiednio 3,161 i 5,092 dla C++ oraz 2,851 i 4,563 dla Pythona. To jest zgodne z przybliżonym wzrostem proporcjonalnym w tych trzech punktach, lecz nie dowodzi dokładnej liniowości. Nie wyznaczano przedziałów ufności ani progu testu hipotezy. Wahania czasu na element i rozstępy pozostają widoczne; nie usuwano skrajnych czasów. Przyczyn odchyleń nie zidentyfikowano.

Wnioski dotyczą konkretnych implementacji, danych, kompilatora i interpretera na tej maszynie. Nie są twierdzeniem o uniwersalnej szybkości języków.

10. Ograniczenia eksperymentu

Poziom A — zapis historyczny

- Jedna maszyna, jedna seria z dwoma blokami, jeden rozmiar i 200 rund. Nie wykonano drugiej pełnej serii B ani badań konfiguracji C-D.
- C++ i Python używają różnych reprezentacji liczb i środowisk wykonania; wspólne jest zadanie i ręczny algorytm, nie instrukcje maszynowe.
- Czas C++ jest krótszy niż 1 ms; narzut i zakłócenia mogą mieć względnie duże znaczenie. Nie kalibrowano rund w osobnym eksperymencie.
- Brak kontroli temperatury, częstotliwości, trybu zasilania i procesów tła. Rozgrzewka nie dowodzi osiągnięcia stabilnego stanu komputera.
- Znana jest zgłoszona rozdzielczość Pythona i nominalny okres C++; nie oszacowano rzeczywistej dokładności ani efektywnej rozdzielczości C++.
- Kompilator może wektoryzować lub inaczej przekształcać pętle. Zgodność checksumu nie dowodzi wykonania każdej instrukcji źródłowej dosłownie.
- Dane są równomiernym multizbiorem po przetasowaniu. Nie reprezentują wszystkich rozkładów ani zastosowań; seed nie jest dowodem jakości losowości.
- Wyniki nie obejmują odczytu, walidacji, wypisywania i startu procesu. macOS/Clang zastępuje szkolny zestaw Windows/MSVC.

Poziom B — zapis historyczny

- B: jedna maszyna, dwie poprawne serie, jedna konfiguracja 100 rund i kontrola ujemna. Historyczne A ma 200 rund; nie łączy się tych wyników.
- Pojedyncze czasy C++ są krótsze niż 1 ms. Nie zwiększano rund ani nie kalibrowano czasu, ponieważ zachowano uzgodnione 100 rund; zakłócenia i narzut mają znaczenie.
- Brak pomiaru temperatury, częstotliwości, procesów tła i efektywnej rozdzielczości C++. Nie potwierdzono identycznych warunków zasilania wszystkich serii.
- Test błędny kończy Python w rozgrzewce, więc nie daje kompletu właściwych czasów; jest to dowód odrzucenia, nie dodatkowy eksperyment wydajnościowy.
- 18 fixture sprawdza wybrane reguły, nie wszystkie możliwe uszkodzenia i awarie. Timeouty i błędy uruchomienia mają obsługę w kodzie, ale nie wymuszano ich w tej serii.
- Atomowa publikacja dotyczy pojedynczego CSV; nie badano przerwania procesu między publikacją CSV a zapisem JSON ani równoczesnej walidacji tego samego katalogu.
- Pominięto ścieżkę instalacji kompilatora, aby metryczka pozostała anonimowa. Nie stanowi to zapisu całego środowiska systemowego.

Poziom C - ograniczenia i odstępstwa

- Jedna maszyna, jedna zaakceptowana seria C1 i jedna C2, po pięć bloków; dodatkowo rzeczywista kontrola ujemna i niepełne przygotowanie C1. Rotacja ogranicza powiązanie wariantu z kolejnością, lecz nie usuwa zakłóceń.
- Nie mierzono temperatury, częstotliwości, obciążenia systemu, cache ani trybu zasilania. Procesy tła mogły wpływać na wyniki; brak danych do wskazania przyczyn.
- C1 stosuje 10 000-elementowy prefiks A, gdyż historyczne A nie miało 10 000 liczb. C2 zachowuje metodę i seed generatora A, lecz jego prefiks różni się od A. To odstępstwo zaakceptowane przez użytkownika; pierwotna tożsamość nie zachodzi.
- Prefixy kontrolują zagnieżdżenie danych, ale wraz z rozmiarem zmienia się skład próbki. Nie jest to badanie wielu niezależnych losowań danych.
- Mierzone są funkcja i pętle, bez I/O i startu procesu. Jednostka ns i metadane zegara nie są pomiarem dokładności. Krótkie próby C++ są podatne na narzut i szum.
- Brak pomiarów Windows/MSVC, innych maszyn i replikacji pełnych serii C. Dodatkowe fixture sprawdzają wybrane reguły, nie dowodzą braku wszystkich błędów.

11. Wnioski

Poziom A — zapis historyczny

Poziom A został wykonany w opisanej adaptacji: wspólny plik ma ponad 10 000 liczb, dwie implementacje wykonują uzgodnioną operację, a wszystkie 20 prób i 8 rozgrzewek zgadzają się z niezależnym wzorcem. Release, granice stopera, źródła i pełne strumienie mają dowody D01-D08. Eksport i kontrola końcowa są w D09.

W tej serii mediany wyniosły 0,752459 ms i 119,905000 ms. Wniosek dotyczy tych implementacji i warunków. Różnice między blokami przemawiają za zachowaniem wszystkich prób i ostrożnością w uogólnianiu. Nie deklaruje się wyników B-D ani przewidywanej oceny.

Najważniejsze decyzje do obrony: jeden plik eliminuje różnice generatorów; multizbiór pozwala sprawdzić wynik prostym wzorem; stoper obejmuje tylko pracę obliczeniową; Release pozwala mierzyć zoptymalizowany C++; rozgrzewki są ustalone z góry; checksum kontroluje poprawność, a SHA-256 tożsamość danych.

Poziom B — zapis historyczny

Poziom B jest potwierdzony przez D10–D16: automat, 18 kontroli walidatora, dwa pełne zaakceptowane przebiegi z CSV, rzeczywiste odrzucenie kopii ze środkiem 0, zachowane źródło błędu i ponowny poprawny przebieg. A zachowuje wyniki i identyfikatory. Na etapie B nie wykonano C–D; nie deklaruje się oceny.

Najważniejsze decyzje do obrony: hash rozpoznaje dane, checksum sprawdza wynik; wzorzec jest niezależny od pętli; rozgrzewki są osobną częścią walidacji; CSV powstaje dopiero po akceptacji całości; rotacja dotyczy bloków; minimum i mediana opisują inne cechy serii; parametrów A i B nie wolno mieszać. Zachowanie 20 020 liczb i zmiana do 100 rund oraz pięć pól są jawnie uzgodnione.

Poziom C - wnioski

D17–D24 dokumentują dwa wybrane badania, harmonogramy przed pomiarami, niezależne wzorce, pełne strumienie, poprawne serie i rzeczywiste odrzucenie. C1 potwierdziło zmianę czasu przy zachowaniu wyniku po zmianie flag kompilacji. C2 pokazało wzrost median z licznością w przybliżeniu proporcjonalny w badanym zakresie. Obserwacje i ograniczenia przedstawiono oddzielnie.

Poziom C wykonano według zaakceptowanej korekty planu. Nie deklaruje się pełnej zgodności z pierwotnym warunkiem identyczności pliku 10 000 z A. Materiały historyczne A–B zachowano; D nie wykonywano. Kompletność dowodów oraz przegląd aktualnego eksportu potwierdza D24, z osobnym wskazaniem zakresu.

12. Wykorzystanie AI i sposób weryfikacji

Poziom A — zapis historyczny

Asystent Codex przeczytał materiał, przygotował plan, kod, zapis dowodów, weryfikator i raport oraz wykonał rzeczywiste uruchomienia. Weryfikacja obejmuje pełny histogram, hash, wzorzec z innej metody, wszystkie rekordy i kody, ponowne rachunki wybranych wierszy oraz przegląd renderu PDF (D07-D09). Nie przypisuje się tych czynności użytkownikowi ani nie deklaruje jego samodzielnej obrony kodu. Przed oddaniem powinien umieć wyjaśnić powyższe decyzje.

Metryczkę zebrano bez nazwy konta i prywatnych ścieżek. Wersja kompilatora zawiera wyłącznie pierwsze dwa wiersze `clang++ --version`, zgodnie z opisem w D02; ścieżki instalacji nie zbierano. Surowych strumieni D03-D06 nie redagowano. Nie commitowano, nie pushowano ani nie wysyłano plików zewnętrznie.

Poziom B — zapis historyczny

W B asystent przygotował plan, automat, walidator, fixture, uszkodzoną kopię, niezależną kontrolę oraz aktualizację raportu; wykonał kompilacje i rzeczywiste uruchomienia. D15 dokładnie wskazuje przeliczone wiersze i statystyki. Nie przypisuje się użytkownikowi tych kontroli ani samodzielnego napisania kodu.

D16 dotyczy historycznego eksportu B, zachowanego w archiwum B; historyczny D09 pozostaje kontrolą eksportu A. Sprawdzenie obejmuje linki i artefakty, hashe kopii źródeł, dane, zgodność wyników, anonimizację oraz render każdej strony. Nie redagowano strumieni raw. Nie commitowano, nie pushowano i nie wysyłano materiałów zewnętrznie.

Poziom C - wykorzystanie AI i weryfikacja

Asystent Codex zbadał zastane pliki, wykrył rozbieżność danych A i przedstawił warianty rozstrzygnięcia. Użytkownik wybrał zachowanie generatora z jawnym odstępstwem. Asystent napisał narzędzia C, wykonał rzeczywiste uruchomienia, fixture i niezależne rachunki, opracował raport oraz obejrzał render PDF. D23 wskazuje dokładne sprawdzone wiersze i działania. Nie przypisuje się uczniowi ręcznych kontroli ani samodzielnego autorstwa tych narzędzi.

Operację traktowano jako uzgodnioną na podstawie polecenia użytkownika; nie zastępowano jej demonstracyjną sumą kwadratów. Przy obronie rozwiązania należy umieć wyjaśnić: checksum a SHA-256, wzorzec niezależny od benchmarków, wyłączenie I/O z pomiaru, kontrolę jednego czynnika, medianę i ograniczenia wniosków. Nie zmieniano surowych dowodów w celu anonimizacji; wersję kompilatora zbierano od początku bez prywatnej ścieżki instalacji. Szkolny PDF i pliki pomocnicze kompilacji/renderowania są wyłączone z zestawu do oddania.